

API

Slide 1 — Cosa sono le API (e come “parlano” via HTTP)

Le **API** (Application Programming Interfaces) sono interfacce che permettono a due programmi di comunicare. Nelle **Web API**, questa comunicazione avviene di solito tramite **HTTP**, usando verbi come **GET** (leggi), **POST** (crea), **PUT/PATCH** (aggiorna) e **DELETE** (cancella). Le API espongono **endpoint** (URL) che accettano richieste e restituiscono risposte, spesso in formato **JSON**.

Nel flusso tipico inviamo una richiesta con **metodo + URL + (eventuali) parametri, header e body**; in risposta otteniamo **status code** (200=OK, 404=non trovato, 401=non autorizzato...), **header** e **corpo** (payload). In Python, la libreria più usata per consumare Web API è `requests`, semplice e leggibile.

```
# Concetti base: un'API HTTP espone endpoint (URL). Noi inviamo richieste e  
riceviamo risposte.
```

```
# Questo è solo un esempio NON eseguito: mostra i pezzi tipici di una richiesta  
a HTTP.
```

```
metodo = "GET"          # Azione (es. GET/POST/PUT/DELETE)  
url = "<https://api.esempio.com/v1/richiesta>"  
parametri = {"q": "python"} # Query string: https://.../richiesta?q=python  
headers = {"Accept": "application/json"} # Cosa vorremmo come risposta
```

```
# La risposta includerà uno status code (200, 404, 500...), header e un body  
(spesso JSON).
```

Slide 2 — Fare una richiesta GET con `requests`

Con `requests` è sufficiente chiamare `requests.get(url, params=..., headers=...)` per ottenere una risposta. Se l'endpoint restituisce JSON, possiamo usare `.json()` per decodificarlo direttamente in dict/list Python. È buona pratica controllare lo **status code** o usare `raise_for_status()` per far emergere subito eventuali errori.

Nel seguente esempio interroghiamo un endpoint pubblico (fittizio) passando una **query string**. Mostriamo lo **status**, un estratto del **JSON** e gestiamo un possibile **KeyError** se il campo non esiste.

```
import requests

BASE_URL = "<https://api.esempio.com/v1/search>" # endpoint fittizio

params = {"q": "python", "limit": 3}          # parametri query
headers = {"Accept": "application/json"}      # preferiamo JSON

resp = requests.get(BASE_URL, params=params, headers=headers)
resp.raise_for_status()                       # solleva eccezione se status >= 400

data = resp.json()                           # dict/list Python dal JSON

print("Status:", resp.status_code)
print("Elementi trovati:", len(data.get("results", [])))

# Accesso sicuro a un campo: usiamo .get per evitare KeyError
for item in data.get("results", []):
    print("-", item.get("title", "<senza titolo>"))
```

Slide 3 — POST, header e autenticazione (token/Bearer)

Molte API richiedono **autenticazione** (es. **Bearer token**). Spesso inviamo dati con **POST** (JSON nel body). Con `requests.post()` passiamo `json=...` per far impostare automaticamente l'header `Content-Type: application/json`. Per autenticarsi, aggiungiamo `Authorization: Bearer <TOKEN>` tra gli header.

Nel codice sotto inviamo un **POST** autenticato: inviamo un payload JSON, controlliamo la risposta e leggiamo l'ID della risorsa creata. In produzione, il token non va hardcoded: meglio leggerlo da **variabili d'ambiente** o da un **segreto** esterno.

```

import os
import requests

API_URL = "<https://api.esempio.com/v1/items>" # endpoint fittizio
TOKEN = os.environ.get("API_TOKEN", "SOSTITUISCI_ME") # leggi token da
env

headers = {
    "Authorization": f"Bearer {TOKEN}",      # autenticazione
    "Accept": "application/json"
}

payload = {"name": "Nuovo elemento", "tags": ["python", "api"]}

resp = requests.post(API_URL, json=payload, headers=headers)
resp.raise_for_status()                    # errore se 4xx/5xx

created = resp.json()
print("Creato con ID:", created.get("id"))

```

Slide 4 — Errori comuni, timeout e retry semplici

Le chiamate a rete possono **fallire** (timeout, DNS, 500 server error). Impostare un **timeout** evita che il programma resti bloccato troppo a lungo; usare `try/except` consente di gestire eccezioni (`requests.exceptions`). Per retry semplici possiamo tentare più volte con un backoff crescente; per scenari seri, valutare `urllib3.Retry` o librerie dedicate.

Nel seguente esempio impostiamo `timeout=5` secondi e facciamo fino a **3 tentativi** in caso di errori temporanei (connessione/timeout o status 5xx). Il **backoff** raddoppia ad ogni tentativo.

```

import time
import requests

URL = "<https://api.esempio.com/v1/ping>" # endpoint fittizio

```

```

MAX_TRY = 3
backoff = 1.0

for attempt in range(1, MAX_TRY + 1):
    try:
        r = requests.get(URL, timeout=5)
        # Consideriamo 5xx come errore temporaneo
        if 500 <= r.status_code < 600:
            raise requests.HTTPError(f"Server error: {r.status_code}")
        r.raise_for_status()
        print("OK:", r.json())
        break
    except (requests.ConnectionError, requests.Timeout, requests.HTTPError) as e:
        print(f"Tentativo {attempt} fallito: {e}")
        if attempt == MAX_TRY:
            print("Ho esaurito i tentativi, interrompo.")
        else:
            time.sleep(backoff) # attesa prima del retry
            backoff *= 2       # raddoppia il backoff

```

Slide 5 — Esporre una piccola API con Flask (Hello JSON)

Oltre a consumarle, con Python possiamo **costruire** API. Un modo semplice è **Flask**: definiamo **route** (endpoint) che ritornano **JSON**. Per casi più strutturati (validazione automatica, schema OpenAPI) vale la pena esplorare **FastAPI**, ma Flask è perfetto per iniziare.

Nel seguente esempio creiamo due endpoint: `GET /health` (per health-check) e `GET /echo?name=...` (ritorna un saluto in JSON). Avvia l'app con `FLASK_APP=app.py flask run` (o `python app.py`) e prova da browser o con `curl`.

```

# pip install flask
from flask import Flask, request, jsonify

app = Flask(__name__)

```

```
@app.get("/health")
def health():
    # Endpoint di controllo: restituisce stato "ok"
    return jsonify(status="ok"), 200

@app.get("/echo")
def echo():
    # Legge un parametro ?name=... dalla query string
    name = request.args.get("name", "mondo")
    return jsonify(message=f"Ciao, {name}!"), 200

if __name__ == "__main__":
    # Avvio dello sviluppo: <http://127.0.0.1:5000/health>
    app.run(debug=True)
```